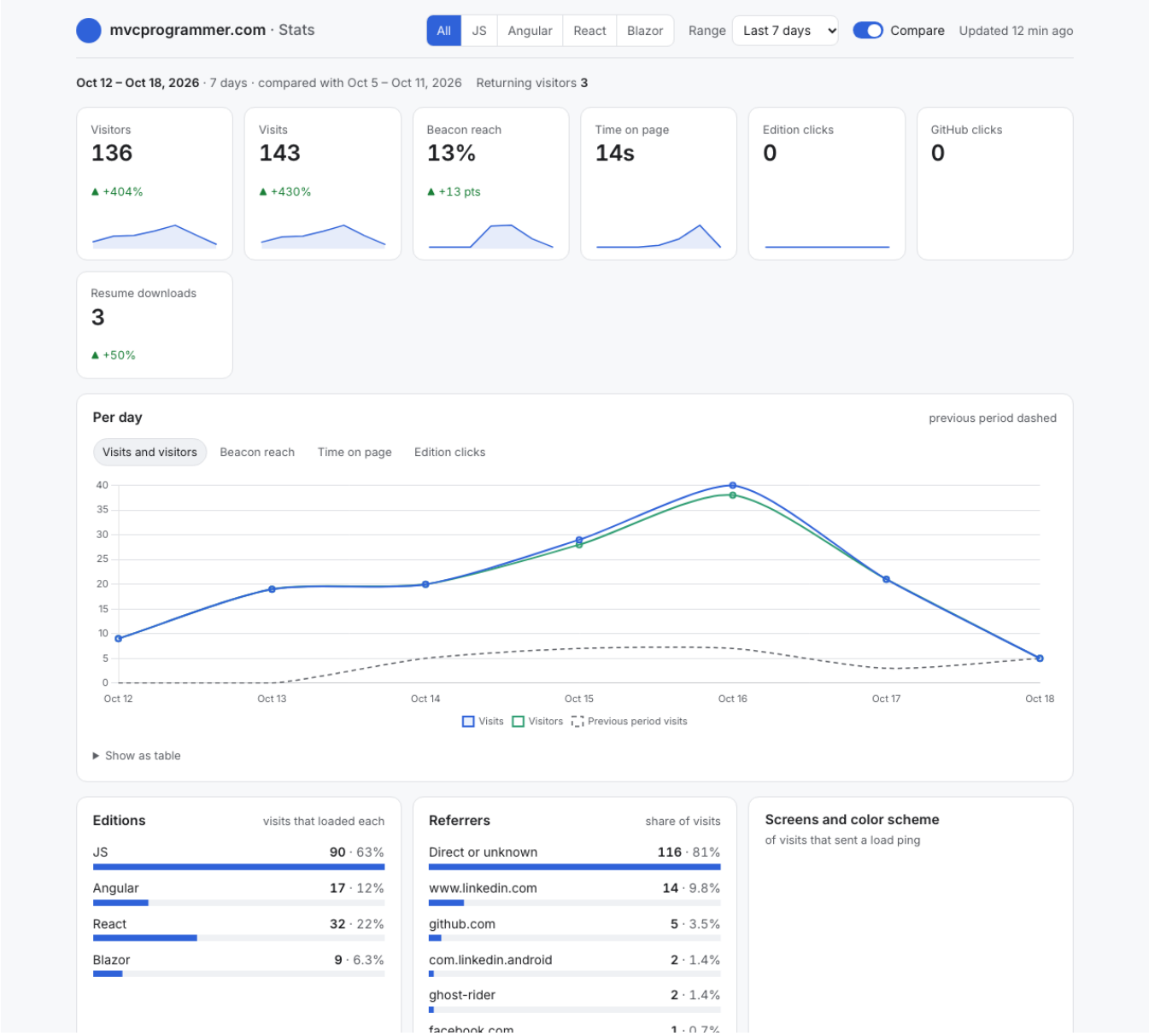
The visitor dashboard

How stats.mvcprogrammer.com is built: the day files exported from the CloudFront access logs, the React page that reads them, what every number means, and the rules for changing, testing, and deploying it.



The dashboard on a laptop, seven days with the previous period compared. The same page on a phone stacks the tiles in two columns and the panels in one.

1. What it is and what it is not

The site has no third-party analytics, by design and by the promise on its privacy page. What it has is a line in the CloudFront access log for every request to all four editions, and a first-party beacon (`GET /ping`) that adds the page load with the screen class and color scheme, the sections a visitor reached, the edition and profile links they followed, and how long the page stayed visible. `tools/cf-stats.mjs` has printed a text report from those logs since October 2026. The dashboard shows the same figures as a page: headline tiles with deltas and sparklines, a daily chart, and breakdowns by edition, referrer, screen, country, section, time on page, and click, with a date range, a previous-period comparison, and click-to-filter segments. It is private, behind a sign-in, and reads static JSON files; there is no server, no database, and no cookie on the public site.

Non-goals

- Not real time. The files are rebuilt hourly from 6 am to 6 pm Eastern and once more at 9 pm; the Updated stamp says when.
- Not a general analytics product. The panels are the ones that answer the site's questions; anything else is a text-report change first.
- Not public. The page and its data sit behind Cognito; a data URL without a session is redirected to sign-in.
- Not a fifth edition. The dashboard wears the shared stylesheet's base JetBrains Rider palette, which none of the public editions uses, and it has no mirror in the other stacks.

2. Where things live

Path	What it holds
`tools/lib/cf-logs.mjs`	The shared parser: log download with the AWS CLI, W3C line parsing, the scanner list, sessions with a 30-minute gap, the visit rule, and the calendar math that cuts days in America/New_York whatever the machine's zone.
`tools/cf-stats.mjs`	The text report, unchanged in output, now a thin caller of the library.
`tools/stats-export.mjs`	The exporter: one JSON day file per day plus an index, from the same rows and visits as the report.
`tools/test/`	Node test-runner tests over a synthetic log (`fixtures/sample-log.mjs`, nine named scenarios) and a golden day file.
`stats/`	The dashboard: React 19, TypeScript strict, Vite, Chart.js 4 through react-chartjs-2, Vitest with Testing Library. Same conventions as `react/`: no CSS-in-JS, no `style` attribute, bundles under `bundles/`, `cssTarget` pinned.
`stats/src/app/pages/Dashboard.tsx`	Loads the days in the range (and the previous period when comparing), derives every number in one `useMemo`, and lays out the tiles, the chart, and the panels.
`stats/src/app/aggregation/`	Pure functions: `filterVisits()`, `summarize()` (the Summary every panel reads), `buildSeries()` (daily, or weekly past 90 days), quantiles, deltas, buckets.
`stats/src/app/query/`	The Query (range, preset, compare, segments) and its round trip through the URL; `useQuery()` binds it to `history`.
`stats/src/app/data/`	The day-file and index types with `SCHEMA_VERSION`, and the fetch client with its in-memory cache.
`stats/src/app/components/`	Tiles, sparkline, bars, panel, table twin with CSV download, the controls, the charts, and the loading, empty, and error states.
`stats/src/app/charts/`	Chart.js registration (only the controllers and elements used) and `useChartTheme()`, which reads the chart colors from the stylesheet tokens through probe classes, so both themes match the page.
`stats/src/stats.css`	Everything the dashboard adds to `styles.css`; the six-color categorical ramp, the up and down colors, and the skeleton tone are `light-dark()` tokens at the top.
`.github/workflows/stats.yml`	The scheduled export: tests, download, export, sync to `stats/data/`, invalidate.
`infra/lambda/stats-auth/`	The Lambda@Edge sign-in check (cognito-at-edge), its tests, and its packaging.
`docs/DEPLOY.md`	The Visitor dashboard section: pipeline secrets and variables, the distribution, Cognito, and the edge function, step by step.

3. How the data flows

```
CloudFront logs (S3)   -->  tools/stats-export.mjs    -->  stats/data/index.json       -->  the page
four prefixes, gzipped      parse, drop scanners,          days/YYYY-MM-DD.json             fetch the range,
W3C lines                   build visits, salt tokens      one record per visit, no IPs     filter, sum, draw
```

The day file

One file per calendar day in the site's time zone, holding the day's request totals and one small record per visit that started that day. Records rather than pre-summed counters, so any combination of filters is exact in the browser, unique visitors inside a filter included, and the files stay small at this site's traffic: about 200 bytes a visit, under 10 KB on a busy day. A day with no log lines at all gets no file and shows as missing, never as a quiet day.

Field	Meaning
`token`	A salted SHA-256 of the IP address and browser string, cut to ten hex digits. The same browser gets the same token every day; nothing can be recovered from it. The salt is a repository secret.
`hour`	The local hour the visit started.
`editions`	The editions it loaded a page on, in display order; a visit that moved between editions lists each.
`referrer`	The external referrer host; absent for a direct visit.
`country, edge`	The two-letter country of the CloudFront edge that served it (`tools/lib/edge-locations.mjs`) and the edge code. Served from, not located in.
`loads, screen, scheme`	From the load ping; absent when the script never ran.
`sections`	The sections reached, in page order.
`seconds`	The largest seconds-visible value the leave ping reported.
`clicks`	The edition, GitHub, and profile links followed, as `kind/target/place`.
`apiCalls, skills, resumes`	Experience API calls, skill-filter clicks by skill, resume downloads by edition tag.

Definitions

These match the text report where it defines a term and tighten it where it does not. Both tools use the same code for every one of them.

Term	Definition
Request	One log line.
Scanner	A request whose user agent matches the bot list, has no user agent, or ended in 4xx; counted in the scanner share and nowhere else.
Visitor	One browser: IP address plus user agent string, carried as the salted token.

Term	Definition
Visit	A visitor's requests with no gap over 30 minutes, which include a page and at least one asset, API call, or ping. The owner's own IP is excluded before anything is counted.
Beacon reach	Visits that sent any ping (load, view, click, or leave), as a share of visits: how many visits ran JavaScript at all. Screen and color scheme come from the load ping alone, which shipped on October 18, 2026.
Engaged visit	A visit with at least one section view ping.
Time on page	The largest leave value a visit reported; shown as the median with the 75th percentile beside it, never as a mean alone.
Section reach	Engaged visits that reached a section, as a share of engaged visits, in page order.
Edition and GitHub clicks	Visits with a click ping of that kind, by target and by the place the link was (header menu, About chips, hero, footer).
Returning visitor	A token seen on more than one day inside the range.
Unique visitors	Distinct tokens among the visits in the range, under whatever filters are active.
Compare	Every headline number against the same-length period immediately before the range, as a signed percentage with an arrow; beacon reach as points.

4. The page

A thin header with the controls, a row of headline tiles, one large chart, then a grid of breakdown panels; the layout Plausible, Fathom, and the GA4 overview share, because it is what people expect from an analytics page. Everything below the header reacts to the controls, and the controls live in the URL (`?range=7d&compare;=1&seg;=edition:react,screen:phone`), so the back button and bookmarks work and the default view has a bare URL.

Control	What it does
Edition switcher	All plus the four editions, as a segmented control; a shorthand for the edition filter.
Range	Today, 7, 30, or 90 days, this month, last month, or a custom pair of dates bounded by the earliest day with data and today.
Compare	Overlays the previous period on the chart as a dashed line and puts a delta on every tile.
Filters	Clicking a row in the Editions, Referrers, Screens, Color scheme, or Countries panel adds a chip; chips are removable one by one or all at once. One filter per dimension; filters combine with AND.
Updated	How long ago the export ran, with the exact time in the tooltip.
Chart point	Clicking a day narrows the range to that day (a week, in weekly mode).

Tiles and panels

Piece	Shows	Empty when
Visitors, Visits	Unique visitors and visit count, with the seven most recent points as a sparkline.	Never; zero is zero.
Beacon reach	Share of visits that sent any ping; greyed when none did.	
Time on page	Median seconds visible; 75th percentile in the tooltip.	
Edition clicks, GitHub clicks	Visits that switched edition, and visits that opened the profile or a repository.	
Resume downloads	A seventh tile, only when the range has any.	
Per day (or per week)	Visits and visitors, or beacon reach, time on page, or edition clicks; previous period dashed; days without a file shaded; weekly from 91 days.	
Editions	Horizontal bars, visits that loaded each; a visit on two editions counts in both.	No visits in this range
Referrers	Top ten hosts with direct grouped, Others collapsed.	No visits in this range
Screens and color scheme	Two small doughnuts with legends that filter.	No beacon data in this range
Countries	Top ten with flags, derived from the edge location, so labelled served from.	No visits in this range
Reading funnel	Bars in page order, share of engaged visits.	No section views in this range
Time on page	Histogram over six fixed buckets with the median and 75th percentile.	No time-on-page data in this range
Edition clicks	By target edition with where the link was, share of beacon-reached visits.	No edition clicks in this range
GitHub and profile clicks	Profile first, then repositories, then LinkedIn and Flickr, with places.	No profile clicks in this range
Time of day	24 columns of visit starts in Eastern time.	No visits in this range
Experience API	Share of visits that loaded it and the skill-filter clicks.	No API calls in this range
Scanner share	Requests against scanner requests per period; collapsed, for operations.	Hidden with no requests

States and accessibility

- Loading: tiles and panels render skeletons in the final layout. Empty: a range with no day files names the earliest and latest available days as links. Partial: days without a file are shaded on the chart and listed in a note. Error: the message and a retry button, never a blank page.
- Every chart has a Show as table twin for screen readers, with a Download CSV button that exports exactly those rows. Bars are inline SVG with attribute widths; deltas carry an arrow so color is never the only signal. Lighthouse accessibility and best practices score 100.
- Light and dark follow the OS through the shared tokens; the charts re-read their colors when the scheme changes. Phone: tiles in two columns, panels stacked, controls behind a Filters button.

5. Running it locally

Node 24, npm, and the AWS CLI with read access to the log bucket (only for the export). All commands run from `stats/`.

```
npm ci
npm run data                    # 90 days of day files into public/data from the cached logs
npm start                       # http://localhost:5173, copies the shared files first
npm run build                   # dist/ (tsc -b, then vite build)
cd .. && node --test 'tools/test/*.test.mjs'   # the parser and exporter
```

Set `CF_STATS_EXCLUDE_IPS` to leave your own visits out of the export, as with the text report. Without any logs, `generateDays()` in `src/testing/fixtures.ts` produces a plausible synthetic month. The dev server has no sign-in; the production page is only ever reached through Cognito.

6. Testing

Three suites, all run by the workflows before anything deploys.

Suite	Runs with	What it proves
`tools/test/` (26)	`node --test`	Parsing and decoding, the window filter, gunzipping a folder, scanner and owner classification, the 30-minute split, a session across local midnight, the visit rule, the command line, the calendar math on both sides of a daylight saving change, every field of a visit record, no IP or browser string in a record, the golden day file byte for byte, determinism, the index merge.
`stats/src/**/*.test.ts(x)` (85)	`vitest run`	Date math, the query round trip, the fetch client (cache, 404 as missing, retry after 500, newer schema refused), every aggregation function, the series (daily, weekly, missing days), the formatters, each component (tiles, sparkline, bars, chips, switcher, picker, updated, table twin, states), the Dashboard over a fake client (numbers, panels, filtering, compare, empty, error), and the App (controls to URL, URL to controls, back button, index failure).
`infra/lambda/stats-auth/` (6)	`node --test`	The options built from the parameters with hardened cookies, a missing parameter refused, the SSM read with decryption, one authenticator per container, and the fail-closed 503 with a retry on the next request.

- Conventions follow the React edition: one test file beside each source file, Arrange, Act, Assert comments, fakes for the dependencies and never for the unit under test, `findBy` or `waitFor` for anything asynchronous.
- Chart.js draws on canvas, which jsdom lacks, so `src/testing/setup.ts` replaces the chart components with a stub that exposes the labels and datasets as data attributes; panel tests check what would have been drawn.
- A schema change bumps `SCHEMA_VERSION` in both the exporter and `types.ts`, regenerates the golden file with `UPDATE_GOLDEN=1`, and is followed by a full rebuild of the day files in the bucket.

7. Deploying and signing in

Two workflows and one distribution. .github/workflows/stats.yml runs every hour from 6 am to 6 pm Eastern and once more at 9 pm (a UTC cron that covers both EDT and EST, with a first job that checks the clock in America/New_York and skips the hours not wanted that day): it tests the parser, downloads the window's logs, rebuilds the last three days (CloudFront delivers some files hours late), syncs them to `stats/data/` with a one-hour browser TTL, and invalidates `/data/*`. The `deploy-stats` job in deploy.yml builds and syncs the page on every push to `main` once `STATS_DISTRIBUTION_ID` is set, and never touches `data/`.

The page is served from the `stats/` prefix of the site bucket by its own distribution with the site's security headers plus `X-Robots-Tag: noindex` and a disallow-all `robots.txt`. A Cognito user pool with one user, a hosted sign-in page, and the `stats-auth` Lambda@Edge viewer-request function keep everything behind a session, the data files included: no cookie means a redirect to sign-in, a valid session proceeds, `/signout` ends it. The function reads its configuration from SSM Parameter Store and fails closed. Every command, id, secret name, and variable is in the Visitor dashboard section of docs/DEPLOY.md; nothing is repeated here.

The `stats/` prefix sits in the same bucket as the public site, and the apex distribution serves that bucket from its root, so a bucket-policy statement denies the prefix to every distribution but the stats one; without it the page and the day files would answer at mvcprogrammer.com/stats/ with no sign-in. The export workflow uploads nothing until the distribution id is set, which is done only after that statement is in place.

Costs: Cognito free under 50,000 monthly users; Lambda@Edge on every request to the distribution (about fifty per page view with the day files), a few thousand a month for one reader at $0.60 per million; kilobytes of S3 and CloudFront; free Actions minutes. Well under a dollar a month.

8. Checklist for a change

Step	Do
1	A new figure starts in `tools/lib/cf-logs.mjs` or the exporter, with a scenario in the fixture log and a test; the text report and the day file see it together.
2	If the day file's shape changes, bump `SCHEMA_VERSION` on both ends, regenerate the golden file, and plan the full rebuild.
3	Derive the number in `aggregation/` as a pure function with a test before touching a component.
4	A new panel is a `Panel` with an empty text, a table twin, and a test over the fake client; a new filterable dimension goes into `query.ts` and `matches()`.
5	Never add a `style` attribute, CSS-in-JS, or an inline script. Colors come from the tokens at the top of `stats.css`; charts read them through `useChartTheme()`.
6	Run `node --test 'tools/test/*.test.mjs'` at the root and `npm test` plus `npm run build` in `stats/`; both green. Check the production stylesheet still carries `light-dark()`.
7	Check light and dark, a phone width, and the empty, partial, and error states.
8	Update this guideline, `stats/README.md`, and the Visitor dashboard section of `docs/DEPLOY.md` when the behavior or the setup changed.
9	Commit on a feature branch, review, then merge to `main` when asked. No AI attribution trailers.

9. Links

What	Where
Live	https://stats.mvcprogrammer.com
Repository	https://github.com/mvcprogrammer/mvcprogrammer.com
Dashboard project	stats/
Dashboard README	stats/README.md
Shared parser	tools/lib/cf-logs.mjs
Exporter	tools/stats-export.mjs
Text report	tools/cf-stats.mjs
Export workflow	.github/workflows/stats.yml
Deploy workflow	.github/workflows/deploy.yml
Sign-in function	infra/lambda/stats-auth/
Deploy walkthrough	docs/DEPLOY.md
Edition guidelines	docs/
Workflow runs	GitHub Actions